Final Report for STAT 451/551

Jack Barney, Kyle Maher, Max Donelan, & Tate Wietfeld

Spring 2026

Introduction

The given FP data contains 200 observations of eight predictors labeld $x1$ through $x8$, and the resulting response parameter y. Additional consideration was given to predicting y_bin which is 1 if y is greater than or equal to 70 and 0 otherwise. $x1$ through $x7$ are continuous while $x8$ is categorical with four groups A, B, C, and D. No context surrounding the predictors or response parameters was given. This report is segmented into initial Exploratory Data Analysis, followed by the Regression Models, then the Classification Models, and concludes with some overall observations. All code can be found in the following GitHub Repository https://github.com/MaxDonelan/STAT-551-Final-Project.

Exploratory Data Analysis

The correlations in Figure 1 show that $x4$ is strongly positively correlated with y and y_bin while $x5$ is strongly negatively correlated with y and y_bin. However, $x4$ and $x5$ are also strongly negatively correlated with each other. This raises concerns for multicolinearity. The plot also shows that $x3$ and $x4$ are correlated. Other than those mentioned there are a few weak correlations, but surprisingly most are around zero.

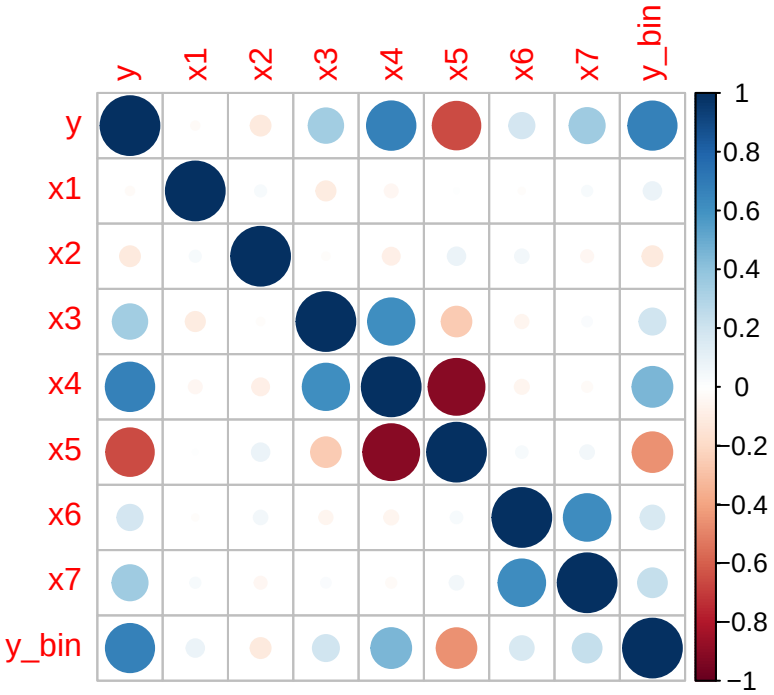

Figure 1: Correlations of Continuous Parameters

The scatterplots in Figure 2 show relatively the same information as Figure 1 with the addition of revealing the $cos(x)$ relationship between $x6$ and $x7$.

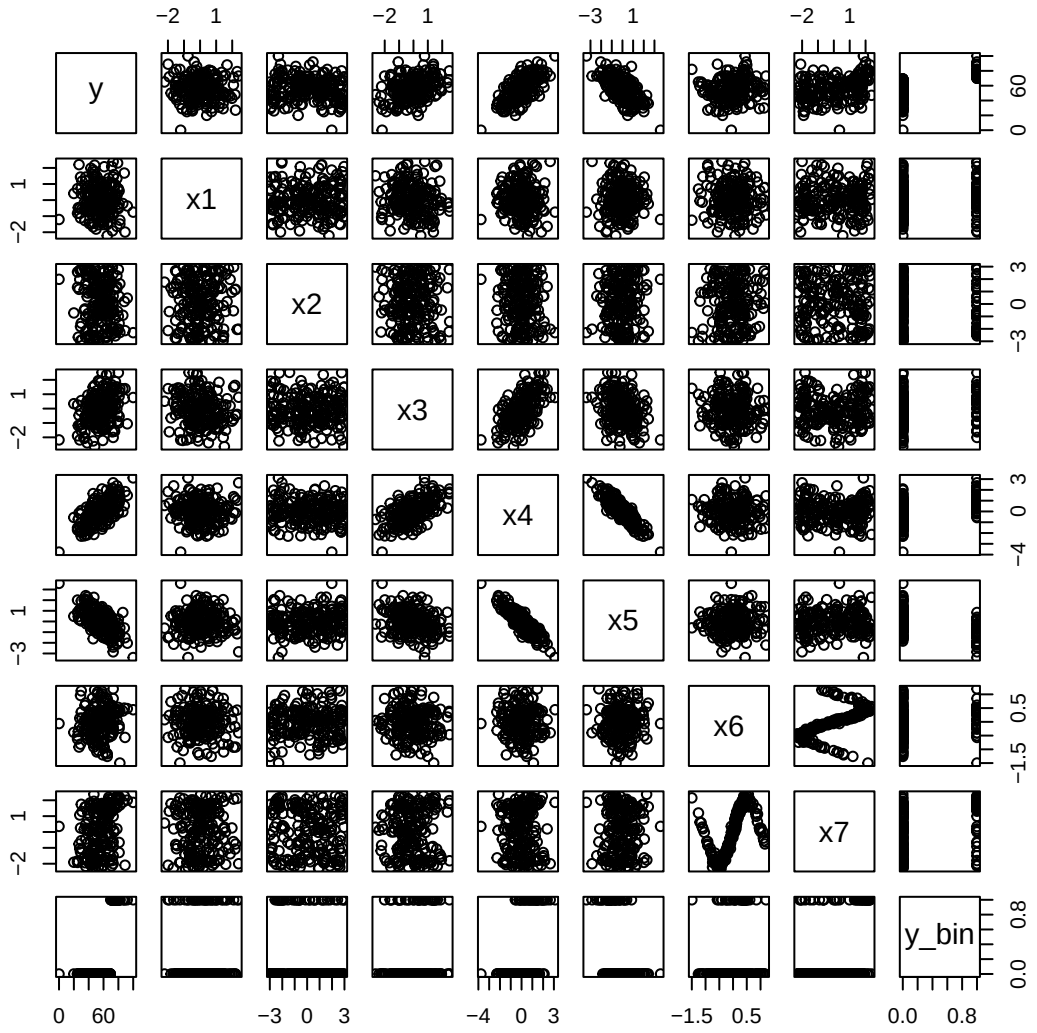

Figure 2: Scatterplots of Continuous Parameters

Our following linear regression and Principal Component Regresison models assume y is normally distributed. We check that here by overlaying a normal distribution with the same mean and standard deviation as y over a histogram of y.

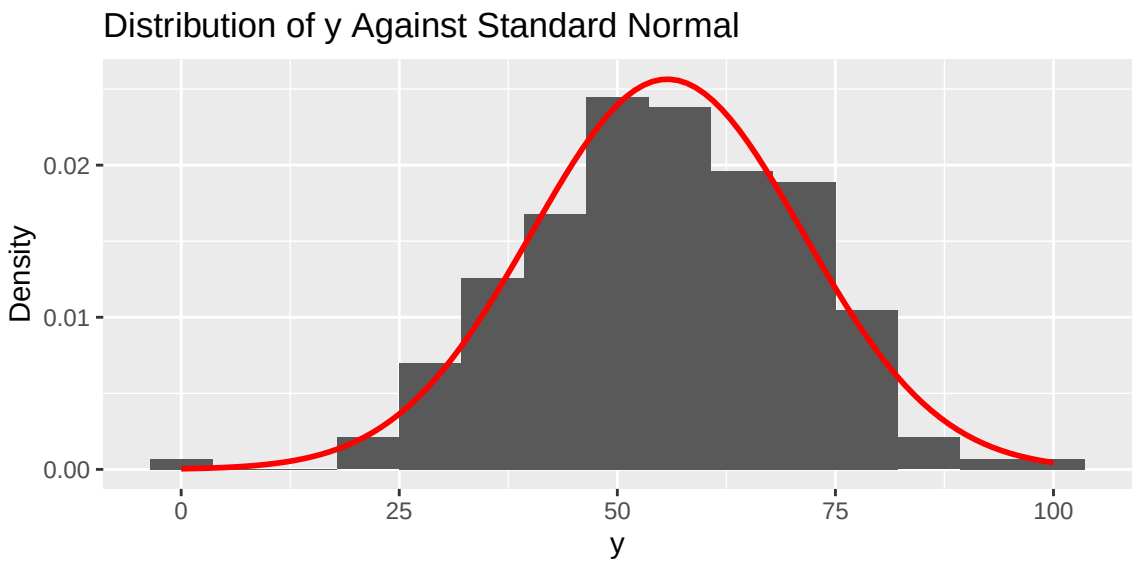

Figure 3: Histogram of y

Here we check the distribution of y grouped by $x8$. Each group doesn't appear as normally distributed as y is overall.

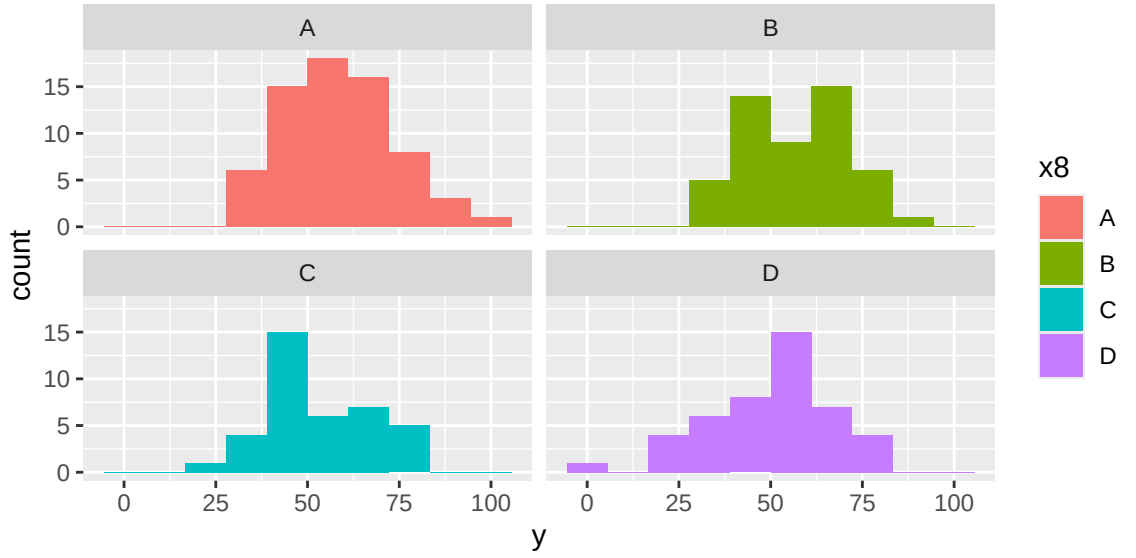

Figure 4: Histograms of y by x8

Similarly we check the relationship between y_bin and $x8$. Here we notice that the classification groups are imbalance which can make prediction more difficult.

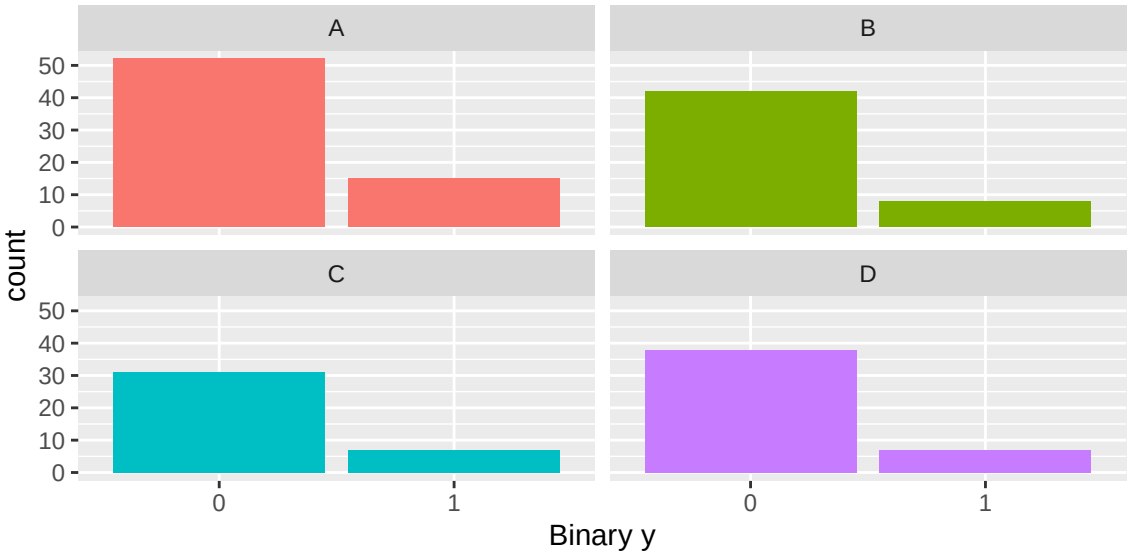

Figure 5: Barplots of binary y by x8

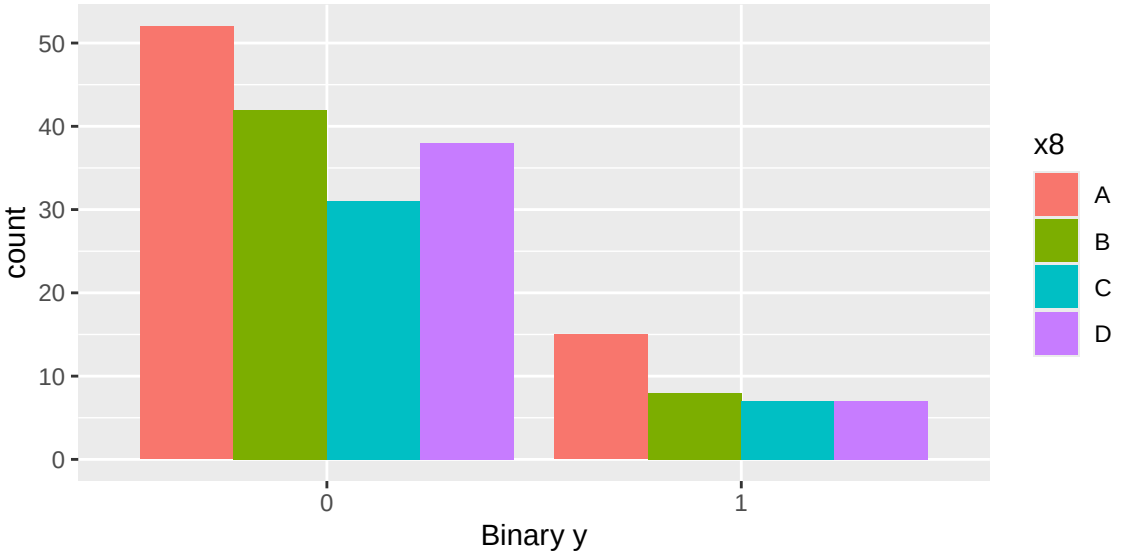

Figure 6: Grouped barplot of binary y by x8

Regression Models

Linear Regression

We fit two basic linear regression models one with all predictors labeled "baseline linear" and another model with only $x4$, $x7$, and $x8$ labeled "small linear".The linear models assume that y is normally distributed which is a reasonable assumption from the EDA plots above. The linear models performed very well in the 10-fold cross validation compared to following regression models.

Random Forest Regression

Random forest is an extension and improvement upon bagging by using decorrelation. Bootstrap aggregation or bagging uses a tree based method and bootstrapping by taking repeated samples from the same training set and taking the average of the models, to make a new model that reduces the variance. The difference between bagging and random forest is that random forest uses, m, a random sample of our predictors from our total predictors p. A majority of the time m is found by evaluating the square root of p. Therefore, when there is a split in the tree, the model is not allowed to consider all the predictors. This is important as it allows other predictors to essentially have a chance, as some of the splits will not consider our strongest predictors. At the end of the models' processes the average of our resulting trees will have less variability and therefore be a more reliable prediction. Random forest is especially helpful when we have many highly correlated predictors.

Boosted Trees Regression

Boosting builds off the premise of bagging with its own twist. In bagging the model is built on the single training set, on the other hand boosting is built sequentially. This means that if a tree is made, the next tree will be grown or made using the information of the previous tree. Any other

tree made after this point will be grown from the information of all the trees before. Thus, boosting is not using bootstrap sampling. This process allows the model to avoid over fitting by learning slowly. Boosting does this by having three tuning parameters the number of trees, a shrinkage parameter, and the number of splits. In general, this model outperforms a random forest model.

Principal Component Regression (PCR)

Principal component regression works by first performing principal component analysis (PCA) to select a subset of principal components, then by using those principal components as the inputs for a linear regression model. PCA uses the singular value decomposition of the data to create a set of principal components, combinations of the original features, that maximize the variance of the data along that axis. To determine the number of principal components to keep, we use the "onesigma" approach given by the selectNcomp() function, which is based on the standard error of the CV results. Using 10-fold cross validation, we obtained a median RMSE of 9.8002 and a median MAE of 7.5488.

Neural Networks

Neural networks perform regression by learning the data via a set of nodes separated into input, hidden, and output layers. For our regression network, we use a single layer model with eight hidden nodes and a reLU activation function. Since Python's machine learning implementations are more developed than R, we chose to use PyTorch and Python 3.14.4 to run our neural networks. For training the model, we use the Adam optimizer as it is implemented in PyTorch with a learning rate of 0.01, 50 epochs of training, a batch size of 32 observations, and a MSE loss function. With 10-fold cross validation, we achieved a median RMSE of 10.4429 and a median MAE of 8.4182.

Plotting the 10-Fold Cross Validation Metrics

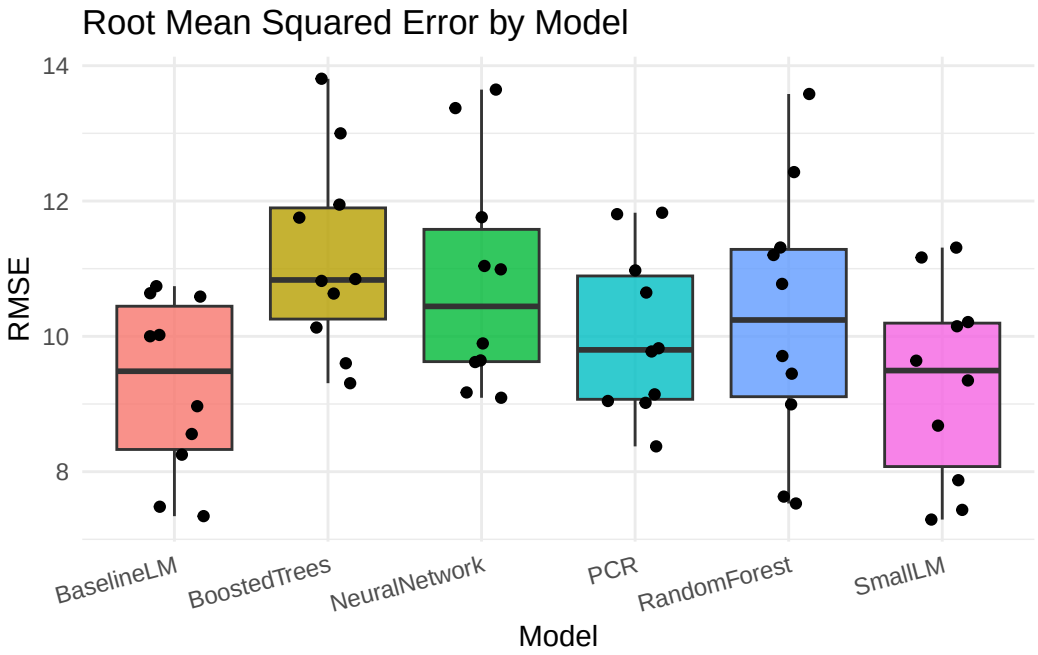

Figure 1: RMSE by Model for Each Fold

A smaller RMSE is better so the Baseline Linear and Small Linear models performed the best here.

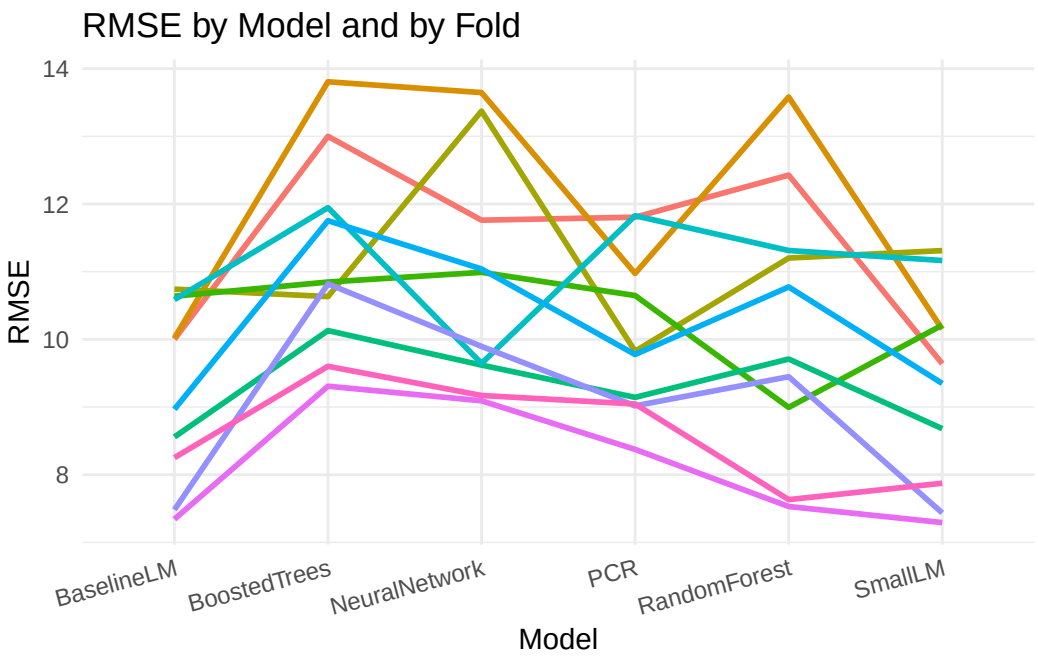

Figure 2: RMSE by Model and by Fold

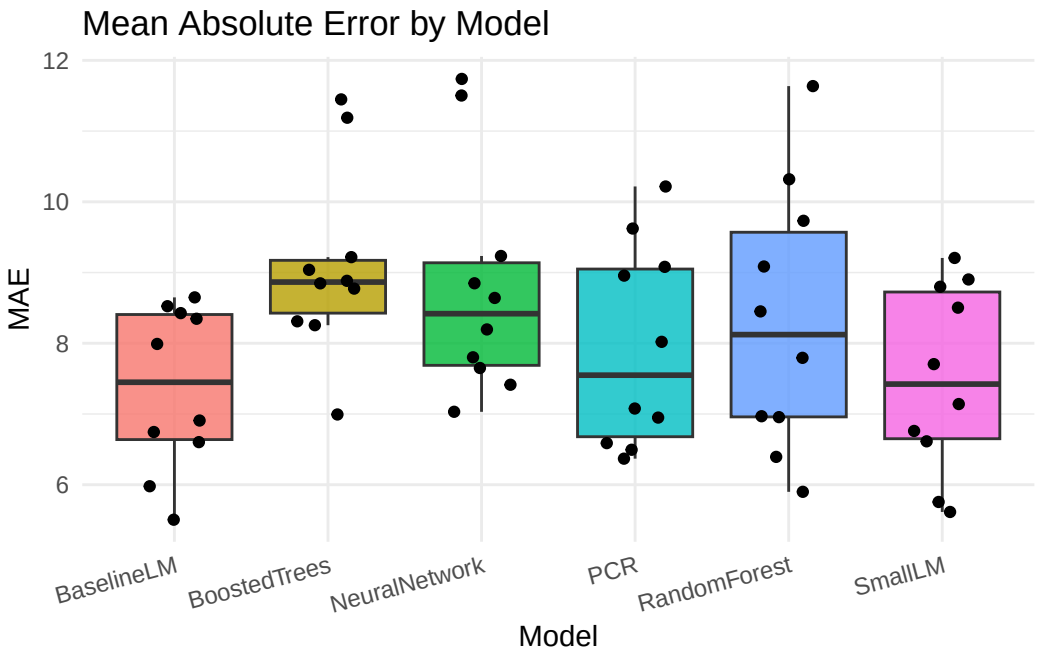

Figure 3: MAE by Model for Each Fold

Classification Models

Standard Binary Logistic Regression

To fit the standard logistic regression, the first step we need to take is to redefine the dependent variable. Previously, y was able to take on any integer value from 0 to 100, but in the case defined by this project, y would need to be reduced to a binary variable depending on its previous larger scale values. This new version of y, ybin, is given a value of 1 if y's previous value was greater than or equal to 70, or 0 otherwise. With variable ybin defined, our next step is to fit the first version of our standard logistic regression model.

Using the glm function in R, we can fit a standard logistic regression using every predictor variable at our disposal (x1 through x8) in efforts to model ybin. The following Figure 1 provides the numerical summary of that model:

```
Call:
glm(formula = ybin ~ x1 + x2 + x3 + x4 + x5 + x6 + x7 + x8, family = binomial,
    data = FP_di)

Coefficients:
            Estimate Std. Error z value Pr(>|z|)
(Intercept)  -2.3626     0.5001  -4.724 2.31e-06 ***
x1            0.5494     0.2742   2.004 0.045122 *
x2           -0.1866     0.1575  -1.185 0.235959
x3           -0.1923     0.7719  -0.249 0.803300
x4            1.7534     1.6982   1.032 0.301851
x5           -0.7956     1.4050  -0.566 0.571203
x6            0.2261     0.5576   0.406 0.685090
x7            0.9472     0.2656   3.566 0.000363 ***
x8B          -1.7534     0.7377  -2.377 0.017461 *
x8C          -0.5180     0.7104  -0.729 0.465833
x8D          -1.8050     0.8173  -2.208 0.027211 *
---
Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1
```

Figure 1: Summary of Initial Logistic Model

Immediately the lack of significant predictors is apparent, with the majority of the independent variables holding p-values greater than 0.05. At this point, we need to check whether we have violated the assumptions of a standard logistic regression. Those assumptions are as follows: The

outcome variable is binary, the data points in each group are independent (little to no clustering), the independent variables must be linearly related to the log-odds of the outcome, there must be little to no multicollinearity, no drastic outliers, and finally that we have a large sample size (Leung, 2025). A few of these we can check off quickly: Our outcome variable ybin is binary, it's defined as described earlier in this section. Next, we can confirm that we have a large sample size because our data has 200 observations, which is generally considered large.

With that, we can begin to critique the other requirements. Starting with checking for outliers and clustering, we can plot the individual predictor variables against the binned y values, whose plots are as follows:

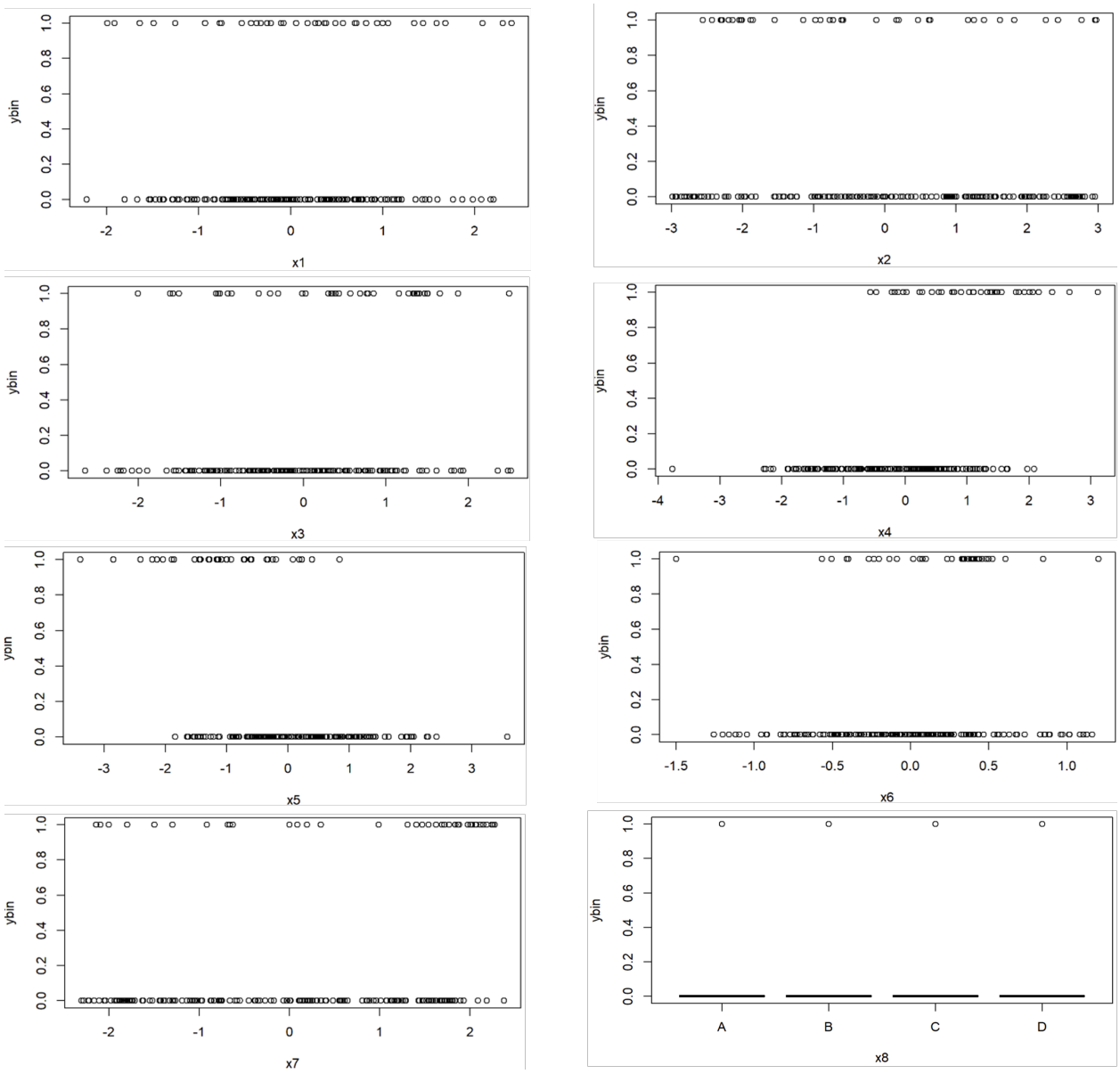

Figure 2: Binned y values vs individual predictors

Seen in Figure 2, none of the values from the predictor variables have extravagant outliers, nor do they cluster dramatically in any scatterplot. At the bottom right of Figure 2 we have the plot of ybin vs x8, which is different from the rest because x8 is a categorical variable. There's no outliers to be seen here either, because values of x8 can only be A, B, C, or D. Thus, we've met two more of our assumptions.

The next assumption we need to check is that our independent variables need to be linearly related

to the log odds of the outcome. This can be done using the Box-Tidwell test. To do so, we must first define a new variable for each predictor variable (xn) that is equal to $xn * log(xn)$. Then, we define a logistic regression model, modeling yin with xn and $xn * log(xn)$ as the only predictors. When we look at the model's summary, if the $xn * log(xn)$'s p-value is significant (< 0.05) , then we cannot assume xn is linearly related to the log odds of the outcome (Leung, 2025). These steps are repeated for each predictor variable, as shown in the below figure. Note that newxn represents the $xn * log(xn)$ value for the nth predictor variable.

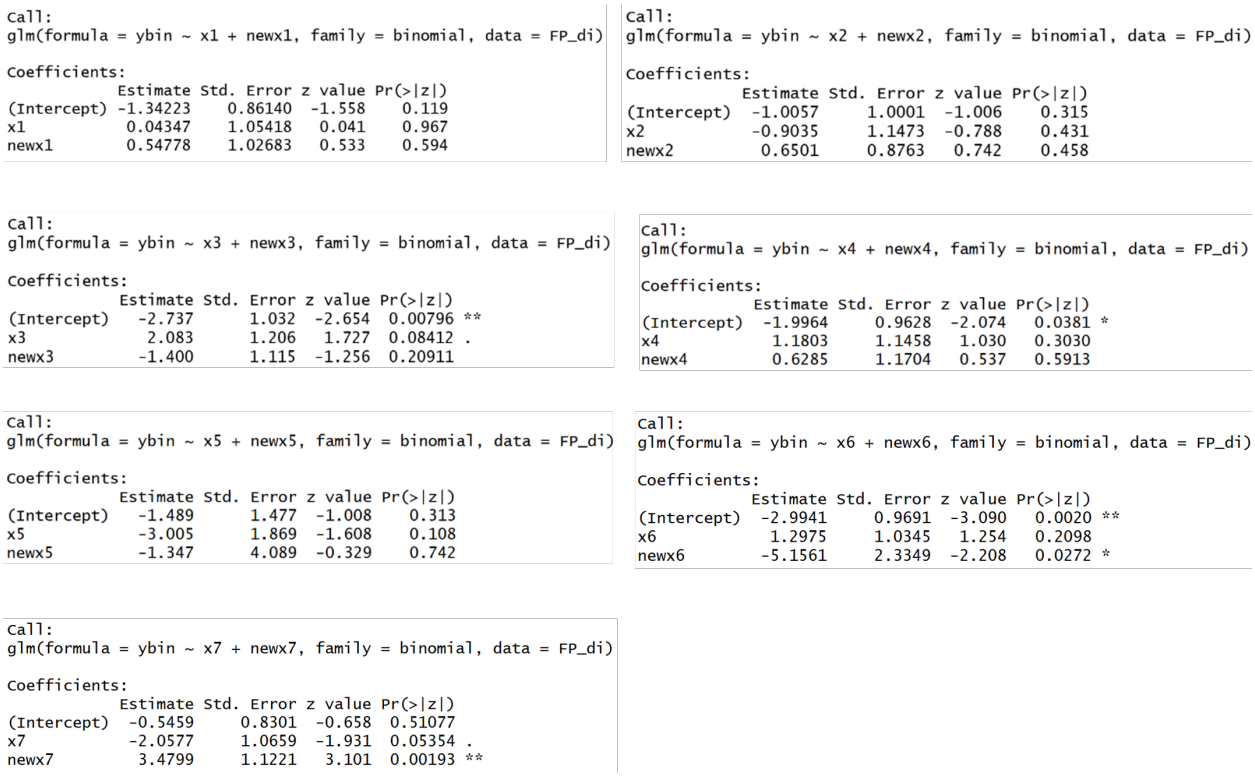

Figure 3: Summaries of Box-Tidwell Models

Above in Figure 3, we notice that for predictors x6 and x7, we have to reject our assumption that they are linearly related to the log-odds of ybin due to their low p-values. This means that for the actual model, we must transform them slightly instead of simply using them as they come from the dataset. In doing so, we are able to meet our assumption of linearity with the log odds of the outcome.

To resolve the final assumption about standard logistic regression models, that there is little to no multicollinearity between predictors, we first need to create a model using all of our predictors. The

only exception to this model is x6 and x7, which will be transformed slightly due to our findings earlier. Upon doing so, we are left with the following model:

```
Call:
glm(formula = ybin ~ x1 + x2 + x3 + x4 + x5 + log(x6 + 3) + log(x7 +
    3) + x8, family = binomial, data = FP_di)

Coefficients:
             Estimate Std. Error z value Pr(>|z|)
(Intercept)  -5.5560     1.6262  -3.417 0.000634 ***
x1            0.5522     0.2683   2.058 0.039555 *
x2           -0.2131     0.1545  -1.379 0.167953
x3           -0.1409     0.7558  -0.186 0.852110
x4            1.7530     1.6602   1.056 0.290992
x5           -0.7173     1.3766  -0.521 0.602309
log(x6 + 3)   0.9907     1.5041   0.659 0.510111
log(x7 + 3)   2.2197     0.6878   3.227 0.001250 **
x8B          -1.5674     0.7053  -2.222 0.026252 *
x8C          -0.4724     0.7020  -0.673 0.501025
x8D          -1.5639     0.7714  -2.027 0.042637 *
```

Figure 4: Improved Standard Logistic Regression Model Summarized

The serious lack of significant predictors is worrying, but a potential solution to that comes from dealing with multicollinearity within the model, if it exists. We can check using the "Cars" package in R, followed by the vif() function, with the above model as the input to it. Doing so garners this result:

```
                GVIF Df GVIF^(1/(2*Df))
x1          1.136026  1        1.065845
x2          1.109377  1        1.053270
x3         11.479255  1        3.388105
x4         27.360025  1        5.230681
x5         17.578727  1        4.192699
log(x6 + 3) 1.378928  1        1.174278
log(x7 + 3) 1.911865  1        1.382702
x8          1.533878  3        1.073903
```

Figure 5: Initial VIF Report

Due to the categorical nature of x8, we don't have exact VIF values, and instead must use the adjusted VIF scores, denoted in Figure 5 as column headed by GVIF^(1/(2*Df)). Adjusted VIF uses the same scale as the normal VIF but is compatible with categorical as well as numeric variables. As seen in Figure 5, there aren't any adjusted VIF measurements above 10, but the lack of significant variables noted in Figure 4 implies that there still may be some issues.

If we drop the highest adjusted VIF variable from the model, x4, and recreate the model, we get the following report from function vif():

	GVIF	Df	GVIF^(1/(2*Df))
x1	1.117639	1	1.057184
x2	1.094885	1	1.046367
x3	1.257233	1	1.121264
x5	1.470021	1	1.212444
log(x6 + 3)	1.379700	1	1.174606
log(x7 + 3)	1.928150	1	1.388579
x8	1.504567	3	1.070455

Figure 6: Improved VIF Report

This is a significant improvement from the previous model, implying minute amounts of multicollinearity. As a result of the nature of multicollinearity, any models with fewer predictors than the model represented in Figure 6 will also have the same or lower levels of multicollinearity. Therefore, our model has kept all the standard assumptions of standard logistic regression intact, and our final standard logistic regression model is as seen in Figure 7:

```
Call:
glm(formula = ybin ~ x1 + x2 + x3 + x5 + log(x6 + 3) + log(x7 +
    3) + x8, family = binomial, data = FP_di)

Coefficients:
             Estimate Std. Error z value Pr(>|z|)
(Intercept)  -5.5091     1.6268  -3.386 0.000708 ***
x1            0.5339     0.2623   2.036 0.041757 *
x2           -0.2057     0.1530  -1.344 0.178817
x3            0.6214     0.2489   2.496 0.012551 *
x5           -2.1350     0.3967  -5.382 7.37e-08 ***
log(x6 + 3)   0.8931     1.5025   0.594 0.552246
log(x7 + 3)   2.2867     0.6810   3.358 0.000786 ***
x8B          -1.6467     0.7050  -2.336 0.019500 *
x8C          -0.4669     0.6988  -0.668 0.504031
x8D          -1.5198     0.7598  -2.000 0.045481 *
```

Figure 7: Summary of Final Standard Logistic Regression Model

This model results in an 87.5% prediction accuracy, leaving room for improvement while still being quite decent at framing the data correctly.

In our final models we test a baseline logistic model using all eight parameters and a log logistic model which uses the log of $x6$ and $x7$ and along with all other parameters except for $x4$.

Regularized Logistic Regression

This version of a logistic regression model adds in a penalty term that works to diminish overfitting. In order to find the error term, we needed to turn the dataset into a matrix and use the cv.glmnet() function to use cross-validation and get a model for potential values. For our error term we used the minimum lambda described by the cross-validation model and implemented it using Lasso (L1) regularization via the glmnet() function using our data in matrix form again. The resulting model returned an accuracy of 88%, which is barely better than that of the standard logistic regression model.

Logistic GAM

Known also as the logistic generalized additive model, is a method for categorizing your outcome variable when you suspect that its predictors don't follow a straight line (Daniel, 2025). We designate such variables by putting them inside the parenthesis of s( ), and using the modeling function gam() from the "mgcv" package. Considering its similar setup to standard logistic regression, we chose to use the same predictor variables as we did in that model, but using the s( ) on every predictor except for x8, because x8 is categorical. This led to a model seen in with an accuracy of 95.5%, a fantastic improvement upon the original logistic regression model.

```
ybin ~ s(x1) + s(x2) + s(x3) + s(x5) + s(log(x6 + 3)) + s(log(x7 +
    3)) + x8

Parametric coefficients:
            Estimate Std. Error z value Pr(>|z|)
(Intercept)   -4.149      1.914  -2.167  0.03021 *
x8B           -1.398      1.151  -1.214  0.22479
x8C           -0.514      1.173  -0.438  0.66126
x8D           -3.373      1.302  -2.592  0.00955 **
---
Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1
```

```
Approximate significance of smooth terms:
                edf Ref.df Chi.sq p-value
s(x1)         2.669  3.316  6.999 0.10236
s(x2)         1.000  1.000  3.589 0.05817 .
s(x3)         8.834  8.978 15.206 0.08415 .
s(x5)         4.869  5.709 19.226 0.00344 **
s(log(x6 + 3)) 1.000  1.000  0.001 0.97595
s(log(x7 + 3)) 6.065  7.161 19.449 0.00777 **
```

Figure 8: Logistic GAM Model Summary

K-Nearest Neighbors

K-Nearest Neighbors (KNN) assigns a class label to a new observation based on the most common class among its closest K neighbors. In our case, $K = 5$ was chosen using cross validation. Before

fitting the parameters $x1$ through $x7$ were scaled to have a mean zero and a standard deviation of one to ensure all parameters contributed relatively equally. The categorical predictor $x8$ was encoded into four binary indicator columns in order to include it in the Euclidean distance measurement. Overall, with 10-fold cross validation KNN achieved a median accuracy of 84.17%, with a false negative rate at 0% for all but one fold.

Neural Networks

Since neural networks can also be used for classification, we applied them here as well. As with the regression neural network, we used a single hidden layer with eight nodes, the Adam optimizer, a batch size of 32, and 50 epochs, all implemented in Python. However, since using the MSE loss function would be nonsensical for classification, we instead used the cross entropy loss function. To this end, with 10-fold cross validation, we achieved a median accuracy of 0.8095, a median false positive rate of 0.1292, and a median false negative rate of 0.375.

Plotting the 10-Fold Cross Validation Metrics

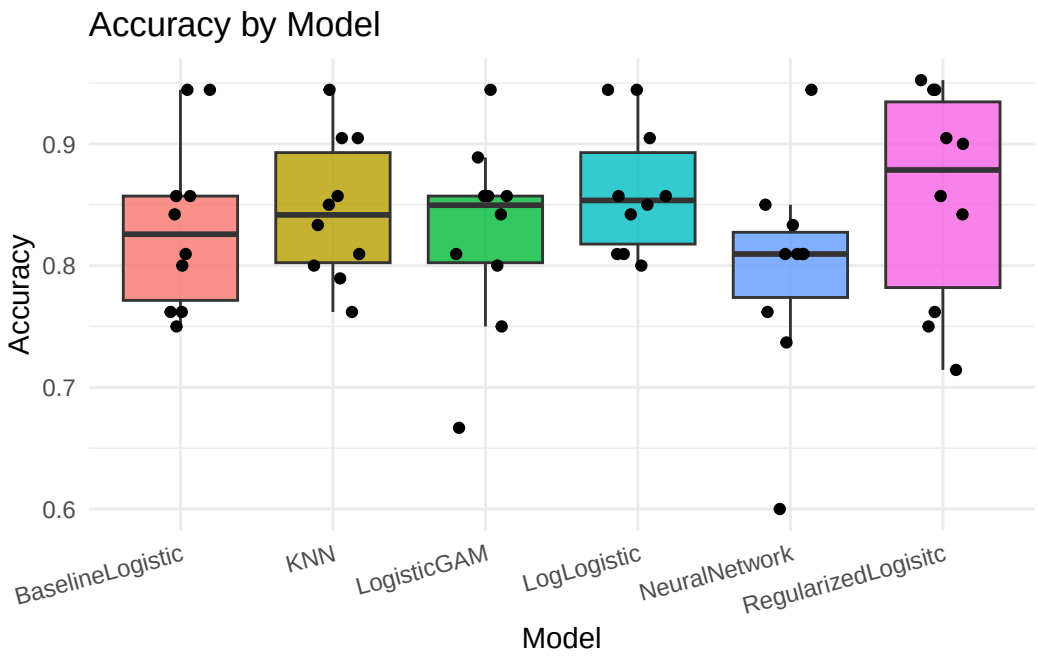

Figure 9: Accuracy by Model for Each Fold

A higher accuracy is better so the KNN, Log Logistic, and Regularized Logisitic perform the best here.

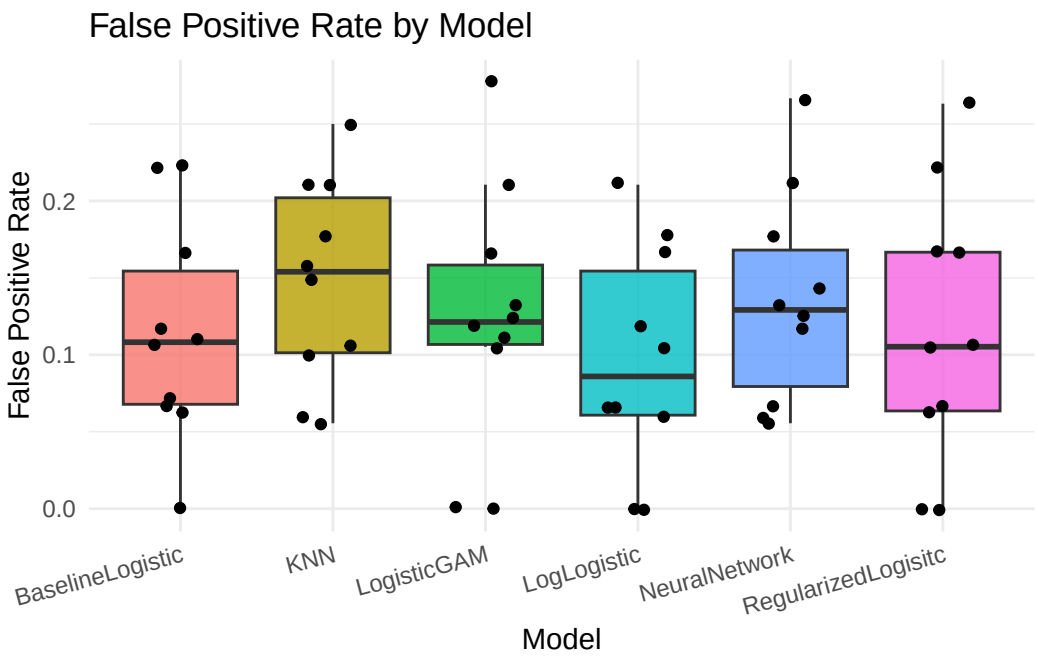

Figure 10: False Positive Rate by Model for Each Fold

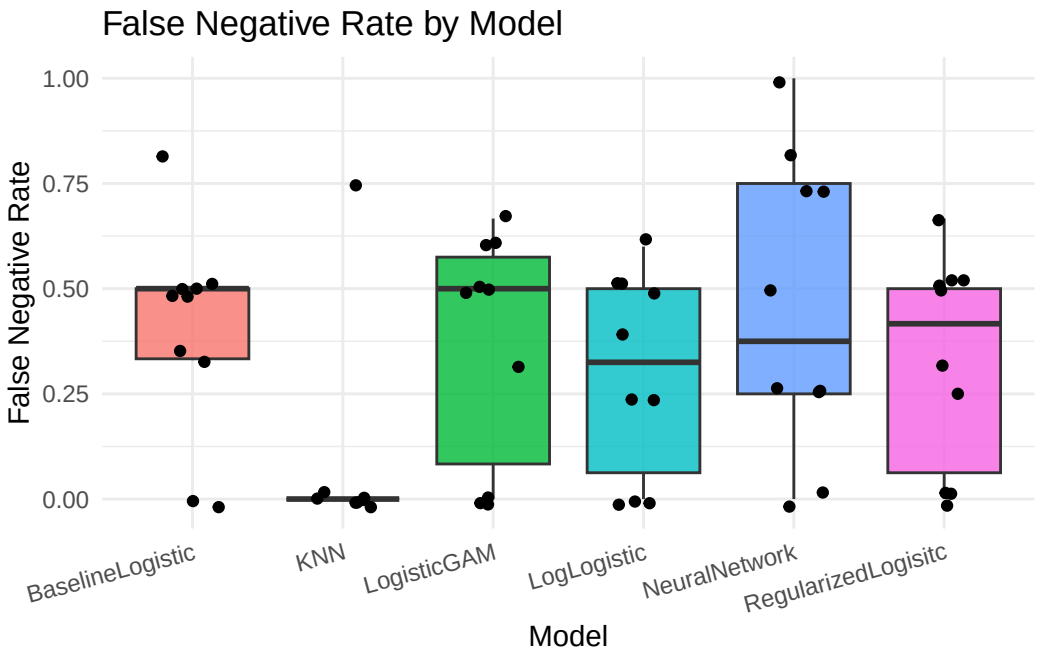

Figure 11: False Negative Rate by Model for Each Fold

This shows that KNN has a larger false positive rate than the other models.

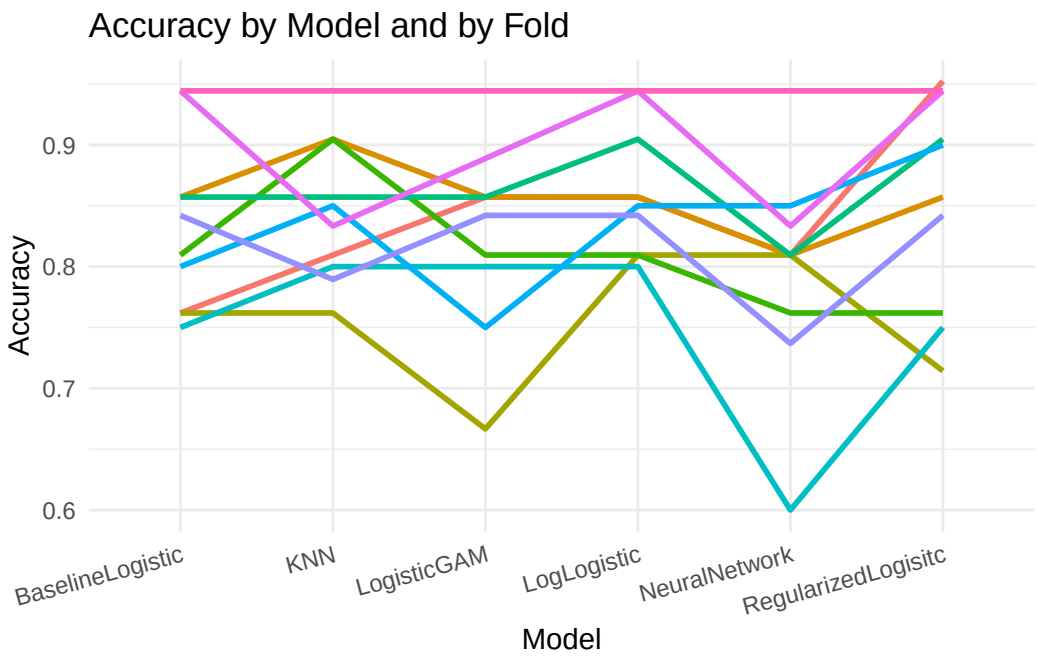

Figure 12: Accuracy by Model and by Fold

This shows that KNN had a significantly smaller false negative rate compared to the other models. The Neural Network model has a high false negative rate which contributes to its fairly low accuracy.

Conclusion

We were given seven numerical predictors labeled $x1$ through $x7$ and one categorical predictor $x8$ for predicting a response variable y. Additionally y was transformed into y_bin by setting y_bin = 1 if $y \geq 70$ and 0 otherwise. Initial exploratory data analysis revealed that predictors $x3$, $x4$, $x5$, and $x7$ were well correlated with y and y_bin, with $x4$ and $x5$ correlated with each other. We applied six regression models to predict y and six classification models to predict y_bin. Then 10-fold cross validation was used to assess the models. Root Mean Squared Error and Mean Absolute Error was assessed for the regression models and Accuracy, False Positive Rate, and False Negative Rate were used to assess the classification models. For regression the linear models performed best while for classification k-nearest neighbors, regularized logistic, and log logistic models all performed well.

References

Daniel. (2025). Understanding Generalized Additive Models. RPubs. https://rpubs.com/AfroLogicInsect/GAM

Leung, K. (2025, March 5). Assumptions of logistic regression, clearly explained. Towards Data Science. https://towardsdatascience.com/assumptions-of-logistic-regression-clearly-explained-44d85a22b290/

Leung, K. (2025, March 5). Assumptions of logistic regression, clearly explained. Towards Data Science. https://towardsdatascience.com/assumptions-of-logistic-regression-clearly-explained-44d85a22b290/

Jerome H. Friedman, Robert Tibshirani, and Trevor Hastie. (2009). The Elements of Statistical Learning. https://hastie.su.domains/ElemStatLearn/

Paszke, A., Gross, S., Massa, F., Lerer, A., Bradbury, J., Chanan, G., … & Chintala, S. (2019). Pytorch: An imperative style, high-performance deep learning library. Advances in neural information processing systems, 32.